SMART COMMUNITY / HACKATHON

Project Brief.
CivicLink.

Un solo spazio per le informazioni della comunità: segnalare, seguire e consultare in modo semplice, strutturato e sempre aggiornato.

PROGETTO	TEMA HACKATHON	DURATA
CivicLink Smart Community	**Smart Community**	**2 giornate**

TEAM

Emanuele Timbo · Pietro Russo · Simone Stellabotte · Davide Storelli

01 / CONTESTO E PROBLEMA

Informazioni disperse.

Nelle piccole comunità (quartieri, condomini, piccoli comuni) le informazioni utili ai cittadini sono sparse tra canali diversi e poco affidabili:

- **Gruppi WhatsApp e Telegram** — messaggi che si perdono
- **Bacheche cartacee fisiche** — non consultabili da remoto
- **Passaparola** — informazioni incomplete o non verificate

TRE CONSEGUENZE CONCRETE

01

Segnalazioni ignorate

Buche, lampioni rotti e rifiuti abbandonati non vengono presi in carico: nessuno sa dove segnalarli in modo strutturato.

NESSUN PUNTO UNICO

02

Oggetti non ritrovati

Chiavi, documenti e oggetti personali persi in luoghi pubblici non trovano il proprietario: l'annuncio resta in un canale che l'interessato non consulta.

ANNUNCI DISPERSI

03

Stato invisibile

Chi segnala non sa se il problema è stato preso in carico, è in lavorazione o è stato risolto.

NESSUN FEEDBACK

TARGET

Residenti di un quartiere o piccolo comune (500–5000 abitanti) che:

- vogliono segnalare problemi del territorio in modo semplice
- vogliono consultare oggetti smarriti/ritrovati nella zona
- vogliono vedere lo stato delle segnalazioni (aperta / in lavorazione / risolta)

Sei obiettivi. Verificabili.

#	OBIETTIVO	COME SI VERIFICA
1	L'utente può inserire una segnalazione con categoria, zona e priorità	Da terminale, opzione 1 → file JSON aggiornato
2	L'utente può consultare tutte le segnalazioni con filtri	Da terminale (opzione 2) e da dashboard HTML
3	L'utente può cambiare lo stato di una segnalazione (Aperta → In lavorazione → Risolta)	Opzione 3 → stato aggiornato nel JSON
4	L'utente può registrare oggetti smarriti o ritrovati	Opzione 4 → file oggetti.json aggiornato
5	I dati sono esportabili in JSON e CSV	Opzione 7 → file in data/exports/
6	La dashboard web visualizza i dati esportati	Apertura frontend/index.html + caricamento JSON

03 / MVP

Tre operazioni. Un ciclo completo.

Abbiamo scelto queste tre operazioni perché coprono l'intero ciclo di vita di una segnalazione: **inserimento** → **consultazione** → **aggiornamento**.

01

Inserire

Perché: senza inserimento non esiste il dato. **Cosa fa:** chiede titolo, descrizione, categoria (Illuminazione / Strade / Rifiuti / Verde pubblico / Sicurezza / Altro), zona e priorità (Bassa / Media / Alta). Salva in data/segnalazioni.json con ID progressivo, data e stato "Aperta". **Validazione:** campi obbligatori non vuoti, valori ammessi.

INSERIMENTO ORDINATO

02

Consultare

Perché: il valore del sistema sta nel rendere i dati visibili a tutti. **Cosa fa:** mostra tabelle leggibili da terminale e, dopo l'export, nella dashboard HTML con filtri per categoria, stato e priorità.

DATI A COLPO D'OCCHIO

03

Aggiornare lo stato

Perché: una segnalazione sempre "Aperta" perde utilità: lo stato trasforma un elenco statico in uno strumento di gestione. **Cosa fa:** dato un ID, cambia lo stato tra Aperta / In lavorazione / Risolta e traccia la data di aggiornamento.

STATO SEMPRE CHIARO

OPERAZIONI SECONDARIE (INCLUSE MA NON CORE)

- Registrazione oggetti smarriti/ritrovati
- Calcolo statistiche (totali, per stato, per categoria)
- Export JSON per la dashboard + export CSV

04 / SCHEMA DATI

Struttura semplice.

DATA/SEGNALAZIONI.JSON

```
{
  "id": 1,
  "titolo": "Lampione non funzionante",
  "descrizione": "...",
  "categoria": "Illuminazione",
  "zona": "Via Roma 15",
  "priorita": "Media",
  "data": "2026-11-06",
  "stato": "Aperta"
}
```

DATA/OGGETTI.JSON

```
{
  "id": 1,
  "nome": "Mazzo di chiavi",
  "descrizione": "...",
  "tipologia": "Chiavi",
  "luogo": "Parco giochi",
  "annuncio": "Smarrito",
  "data": "2026-11-06",
  "stato": "Attivo"
}
```

DATA/EXPORTS/DASHBOARD_DATA.JSON

Export unificato. Contiene: **meta, statistiche, segnalazioni, oggetti**. Letto dal frontend tramite FileReader.

05 / PIANO DI LAVORO

Fasi, priorità e responsabili.

FASE	ATTIVITÀ	PRIORITÀ	RESPONSABILE
Ideazione	Scelta tema, definizione problema, brief	Alta	Emanuele Timbo
Setup	Struttura cartelle, moduli Python, README	Alta	Pietro Russo
Backend core	main.py, segnalazioni.py, oggetti.py, validazione input	Alta	Simone Stellabotte
Gestione dati	gestione_file.py, JSON, CSV, backup	Alta	Emanuele Timbo
Statistiche	statistiche.py, calcolo indicatori	Media	Simone Stellabotte
Frontend	index.html, style.css, script.js, FileReader	Alta	Pietro Russo
Test	Casi normali, di confine, di errore; ritest dopo fix	Alta	Tutti
Documentazione	Kanban, checkpoint, report finale	Media	Davide Storelli
Presentazione	Demo live, script, risposte individuali	Alta	Tutti

DIPENDENZE

- Il frontend dipende dall'export JSON del backend
- I test dipendono dal completamento del backend
- La presentazione dipende da tutto il resto

06 / SCELTE TECNICHE

Scelte motivate.

SCELTA	ALTERNATIVA SCARTATA	MOTIVAZIONE
JSON come formato primario	Solo CSV	JSON preserva la struttura annidata (statistiche + liste), più adatto alla dashboard
CSV come formato secondario	—	Richiesto dalla traccia, utile per analisi esterne
Python standard library	Flask, Django	Vincolo hackathon: nessun server
HTML/CSS/JS vanilla	React, Vue, Bootstrap	Vincolo hackathon: nessun framework
FileReader API per caricamento	Fetch da server	Vincolo hackathon: 100% offline, nessun server
Backup .bak prima di salvare	Salvataggio diretto	Previene perdita dati in caso di scrittura interrotta

07 / FUORI DALL'MVP

Cosa non c'è (e perché).

FUNZIONALITÀ	PERCHÉ ESCLUSA
Login e autenticazione	Non richiesta dalla traccia; aggiungerebbe complessità senza valore per la demo
Notifiche email/push	Richiederebbe server e API esterne
Mappa interattiva	Richiederebbe librerie esterne (Leaflet, Google Maps)
Commenti sulle segnalazioni	Scope troppo ampio per 2 giornate
Database	Vincolo hackathon: solo file

Rischi sotto controllo.

RISCHIO	MITIGAZIONE
JSON corrotto da scrittura interrotta	Backup .bak automatico prima di ogni salvataggio
Input utente malformato	Validazione in _input_valido() e _input_non_vuoto()
Dashboard non carica il file	Controllo struttura JSON + toast di errore
Tempo insufficiente per il frontend	Dashboard minimale funzionante (tabelle + indicatori) come fallback

CHECKLIST DI CONSEGNA

- [x] **Problema preciso e target concreto**
- [x] **6 obiettivi verificabili**
- [x] **3 operazioni MVP motivate**
- [x] **Schema dati definito**
- [x] **Piano con attività, priorità e responsabili**
- [x] **Scelte tecniche motivate**
- [x] **Funzionalità escluse dichiarate**
- [x] **Rischi e mitigazioni**